

Chapter 4: Complete Folder Structure & File Deep Dive

Yeh chapter aapke system ka GPS hai. Interviewer jab aapse puchega "Tumne `rag.service.js` ko controllers folder mein kyun nahi rakha?", toh yahan se aapko exact reason, best practices, aur industry standards milenge.

1. The Big Picture (Root Folder Structure)

Ek standard Monorepo ya separate backend/frontend setup mein folders ko clearly divide karna sabse zaroori hota hai taaki developers easily navigate kar sakein.

```
graph TD
    Root[AI Study Buddy Project] --> FE[Frontend: /src]
    Root --> BE[Backend: /backend]

    subgraph Frontend (/src)
        FE --> Components[/components/]
        FE --> Pages[/pages/]
        FE --> Store[/store/]
        FE --> Services_FE[/services/]
        FE --> Lib[/lib/]
    end

    end

    subgraph Backend (/backend)
        BE --> Config[/src/config/]
        BE --> Routes[/src/routes/]
        BE --> Controllers[/src/controllers/]
        BE --> Services_BE[/src/services/]
```

```
BE --> Models[/src/models/]
BE --> Middlewares[/src/middlewares/]
BE --> Utils[/src/utils/]
BE --> Data[/data/]
end
```

🌸 2. Frontend Folder Breakdown (/src)

Frontend **React (Vite)** par based hai.

src/components/ui/

- **What it is:** Saare reusable dumb/presentational components (Buttons, Cards, Inputs).
- **Industry Standard:** UI libraries (jaise shadcn/ui ya Material-UI) yahi pattern follow karti hain. Design system ek hi jagah hona chahiye.

src/pages/

- **What it is:** Smart components jo ek route (URL) ko represent karte hain (e.g., `Dashboard.jsx`, `Chat.jsx`).
- **Best Practice:** Pages mein sirf layout aur state integration hoti hai. UI elements hamesha `components/` se aate hain.

🌸 Deep Dive: Important Frontend Files

1. **src/store/index.js (Zustand Store)**

- **Purpose:** Pure app ki global state (Auth, Chat, Notes) ko ek jagah manage karna.
- **Why it exists:** React mein agar data deep components tak bhejna

ho (Prop Drilling), toh code messy ho jata hai. Global store iska solution hai.

- **Dependencies:** `zustand`.
- **Flow:** User clicks login -> `authSlice` ki action call hoti hai -> API hit hoti hai -> State update hoti hai -> React re-render hota hai.
- **Interactions:** Baaki sabhi `.jsx` files isko `useAuth()`, `useChat()` hook ki tarah use karti hain.
- **Best Practices:** Slices pattern (`authSlice`, `chatSlice` alag-alag banakar ek jagah merge karna).
- **Possible Improvements:** Persist middleware lagana taaki refresh karne par state (jaise dark mode) lost na ho.

2. `src/services/_adapter.js`

- **Purpose:** API requests ko Mock data aur Real Backend ke beech toggle (switch) karna.
- **Why it exists:** Frontend developers ko UI banane ke liye backend ke live hone ka wait na karna pade.
- **Dependencies:** `axios`, `import.meta.env.VITE_USE MOCK`.
- **Flow:** Frontend kisi endpoint (`fetchNotes`) ko call karta hai. Yeh file check karti hai agar `VITE_USE MOCK` true hai, toh fake JSON bhej deti hai, warna asli Axios request.
- **Industry Standard:** E-commerce aur SaaS platforms mein isey "Adapter Pattern" ya "Interface Segregation" kehte hain.

3. `src/lib/axios.js`

- **Purpose:** Axios client ka global configuration, base URL, aur Interceptors setup karna.
- **Why it exists:** Har API call se pehle automatically "Authorization" header mein JWT token daalne ke liye.
- **Flow:** API Call initiate hoti hai -> Axios Request Interceptor token lagata hai -> Server ko Request jati hai. Agar 401 error aaye, toh Response Interceptor user ko logout kar deta hai.

🔧 3. Backend Folder Breakdown (/

backend/src)

Backend **Node.js**, **Express**, aur **MongoDB** par based hai, aur **Controller-Service-Model** pattern strictly follow karta hai.

src/routes/

- **What it is:** Express routers jo URLs (e.g., `POST /api/chat`) ko controllers se map karte hain.
- **Best Practice:** Router mein sirf path aur middleware lagne chahiye, koi business logic nahi.

src/models/

- **What it is:** MongoDB schemas (Mongoose). Database structure define karta hai.

✧ Deep Dive: Important Backend Files

1. backend/src/app.js & server.js

- **Purpose:** App configuration aur Server boot-up.
- **Why it exists:** `server.js` sirf port listen karta hai aur DB connect karta hai. `app.js` middlewares (CORS, Rate Limiter) aur routes setup karta hai. Inko alag rakhne se API testing (Supertest) aasan ho jati hai kyunki port block nahi hota.
- **Flow:** Node boot -> `env.config` load -> `db.config` connect -> Middlewares apply -> Routes mount -> Port 5000 listen.
- **Best Practices:** `app.js` ke end mein `HAMESHA globalErrorHandler` middleware hona chahiye.

2. backend/src/controllers/chat.controller.js

- **Purpose:** HTTP request (params, body) ko receive karna aur

response (JSON) bhejna.

- **Why it exists:** Separation of Concerns (SoC). API interface ko logic se alag rakhna.
- **Dependencies:** `chat.service.js`, `catchAsync.js` (for error handling).
- **Flow:** Request aati hai -> Body se `query` nikalta hai -> `chat.service.sendMessage()` ko call lagata hai -> Success response bhejta hai.
- **Possible Improvements:** Input validation controllers ke bajaye route level par middleware (Zod) se karna (jo humne is app mein properly apply kiya hai).

3. `backend/src/services/rag.service.js`

- **Purpose:** Retrieval-Augmented Generation pipeline ka "Dimagh". Text chunking, vector search, aur Prompt engineering sab yahin combine hota hai.
- **Why it exists:** RAG flow bahut complex hai. Isey controllers se bahar rakhna zaroori hai.
- **Dependencies:** `embeddings.service.js`, `groq.service.js`.
- **Flow:**
 1. User ka question aaya.
 2. `embeddings.service` se uske related chunks vectors compare karke nikaale.
 3. System Prompt banaya: "Answer using this context: [Chunks]".
 4. Groq API ko call kiya and answer return kiya.
- **Best Practices:** Single Responsibility Principle. Yeh file strictly orchestration ka kaam karti hai.

4. `backend/src/middlewares/error.middleware.js`

- **Purpose:** Pure backend mein aane wale saare errors ko pakadna aur unhe uniform format mein frontend ko bhejna.
- **Why it exists:** Agar database crash ho jaye, toh user ko ek ganda HTML error ya app crash nahi milna chahiye. Use ek clean `{"error": "Something went wrong"}` milna chahiye.
- **Flow:** Kisi bhi controller mein Error throw hota hai -> `CatchAsync`

usko next() mein daal deta hai -> `error.middleware.js` us error ko pakadta hai -> Format karta hai -> Response send karta hai.

- **Industry Standard:** Production aur Development errors ko alag tarike se handle karna. Dev mein pura Stack Trace dikhana, Prod mein internals hide karna.

5. `backend/src/models/Message.model.js`

- **Purpose:** Chat messages ka blueprint.
 - **Why it exists:** Conversation history ko store karne ke liye.
 - **Dependencies:** `mongoose`.
 - **Flow:** `Chat` id ko refer karta hai.
 - **Best Practices: Reference Pattern.** Humne messages ko `Chat` model ke andar "embed" nahi kiya, kyunki MongoDB mein 16MB document size limit hoti hai. Lamba chat history document crash karwa deta.
 - **Possible Improvements:** `timestamp` aur `chatId` par index aur fast banaya ja sakta hai pagination ke liye.
-



Generated Interview Questions

1. "Tumhare backend mein `app.js` aur `server.js` alag-alag files kyun hain? Ek mein kyun nahi likha sab?" *Answer:* "Testing ke liye. Jab hum Jest aur Supertest se integration tests likhte hain, toh humein Express `app` object chahiye hota hai bina usko specific port par listen karwaye. Agar ek hi file mein hoga, toh tests chalate waqt 'Port in use' error aayega. Isliye `app.js` sirf `app` banata hai, aur `server.js` usko run karta hai."
2. "Agar main kisi API call ke response structure ko change kar doon, toh tumhare architecture mein kitni jagah code change karna padega?" *Answer:* "Backend mein sirf `apiResponse.js` utility (jo har controller use karta hai) mein change karna padega. Frontend par, kyunki maine `_adapter.js` (Adapter Pattern) lagaya hai, mujhe sirf adapter update karna padega aur saare

React components ko lagatar correct data milta rahega. Yeh Loose Coupling ka fayda hai."

3. **"Zustand ka use karke tumne Prop Drilling toh rok li, par kya isse unnecessary re-renders nahi hote?"** *Answer:* "Nahi, kyunki Zustand selectors allow karta hai. Main component mein sirf wahi state ka tukda (slice) fetch karta hoon jo chahiye (e.g., `const user = useAuth(state => state.user)`). Agar chat history update ho rahi hai, toh mera navbar re-render nahi hoga."
 4. **"Tumne `error.middleware.js` banaya. Lekin Controller mein agar Async function fail hua toh Express by default crash ho jata hai. Usey kaise handle kiya?"** *Answer:* "Maine ek Higher Order Function banaya hai jiska naam `catchAsync` hai. Main apne saare controllers ko uske andar wrap karta hoon. Yeh function async code se nikle huye har Promise rejection ko pakad kar explicitly `next(err)` call kar deta hai, jo seedha mere global error middleware tak chala jata hai."
-

⚡ Quick Revision Notes

- **Frontend Folders:** `/ui` (Dumb design), `/pages` (Smart routing), `/store` (Zustand Global State), `/services` (Mock vs Real APIs).
- **Backend Folders:** `/controllers` (HTTP Logic), `/services` (Business/AI Logic), `/models` (DB Architecture), `/middlewares` (Security, Logs, Errors).
- **Patterns Used:**
 - **Adapter Pattern** (Frontend `_adapter.js`)
 - **Controller-Service Pattern** (Backend SoC)
 - **Reference Pattern** (MongoDB Models like Messages)
- **Key Concepts:** `CatchAsync`, Global Error Handler, Axios Interceptors, Loose Coupling.